

Báo cáo Midterm Project

Họ và tên: Nguyễn Thị Thanh Trúc
MSSV: 202417052
Ngày: 15/04/2026

Assignment A:

10. Nhập số nguyên dương N từ bàn phím có từ 2 chữ số trở lên. In ra màn hình số được tạo bởi các chữ số của N theo chiều ngược lại.

→ Yêu cầu đề bài:

- Nhập số nguyên dương N (có ít nhất 2 chữ số), ví dụ: 12345.
- In ra màn hình số đảo ngược: 54321.

→ Phân tích cách thực hiện:

- **Ý tưởng:** Muốn lấy từng chữ số từ phải sang trái của một số nguyên N, ta thực hiện phép chia N cho 10.
 - Phần **dư** của phép chia $N \bmod 10$ là **chữ số cuối cùng** của N.
 - Sau đó, gán $N = N / 10$ để **loại bỏ chữ số cuối** vừa lấy.
 - Lặp lại quá trình này cho đến khi $N = 0$, ta sẽ lần lượt thu được các chữ số của số ban đầu theo thứ tự từ phải sang trái.
- **Tóm lại:** $(N \% 10) \Leftrightarrow$ Lấy chữ số cuối && $(N / 10) \Leftrightarrow$ Bỏ chữ số cuối.

→ Code + giải thích:

```
.data
msg1: .asciz "Nhập N: "          #Chuỗi thông báo nhập số
msg2: .asciz "Số đảo ngược là: "  #Chuỗi in kết quả
.text
main:
# In câu dẫn:
li a7, 4      # mã dịch vụ a7 = 4 → in chuỗi ra màn hình
la a0, msg1   # a0 = địa chỉ của chuỗi msg1
ecall        # gọi hệ điều hành in chuỗi msg1: "Nhập N: "
# Nhập số N
li a7, 5      # mã dịch vụ a7 = 5 → nhập số nguyên
ecall        # gọi hệ điều hành: nhập N từ bàn phím, lưu vào thanh ghi a0 hay a0 = N
```

```
li t0, 0 # t0 = reverse = 0 → khởi tạo số đảo: sau mỗi vòng lặp  $t0 = t0 * 10 + n \% 10$ 
li t2, 10 # t2 = 10 → dùng để chia 10 và nhân 10
```

reverse_loop:

```
beq a0, zero, print_result # nếu a0 = N = 0 thì dừng → nhảy tới nhãn print_result
# nếu a0 = N != 0 thực hiện tiếp câu lệnh bên dưới:
rem t1, a0, t2 # t1 = a0 % 10 → lấy chữ số cuối cùng
div a0, a0, t2 # N = N / 10 → bỏ chữ số cuối cùng
```

```
mul t0, t0, t2 # t0 = t0 * 10
```

```
add t0, t0, t1 # t0 = t0 + t1 = t0 + (N % 10)
```

→ $t0 = reverse = t0 * 10 + (N \% 10)$ → cập nhật số đảo

```
j reverse_loop # quay lại đầu vòng lặp
```

print_result:

In thông báo

```
li a7, 4 # mã dịch vụ a7 = 4 → in chuỗi ra màn hình
```

```
la a0, msg2 # a0 = địa chỉ của chuỗi msg2
```

```
ecall # gọi hệ điều hành in chuỗi msg2: "Số đảo ngược là: "
```

In số đảo

```
mv a0, t0 # move t0 → a0: a0 = giá trị đảo để in ra màn hình
```

```
li a7, 1 # mã dịch vụ a7 = 1 → in số nguyên ra màn hình
```

```
ecall # in số đảo
```

Kết thúc

```
li a7, 10 # mã dịch vụ a7 = 10 + ecall → kết thúc chương trình!
```

```
ecall
```

→ Tổng quan chương trình: Chương trình thực hiện ba bước chính:

1. Nhập dữ liệu

In lời nhắc và dùng syscall để đọc số nguyên dương N từ bàn phím vào thanh ghi a0.

2. Đảo các chữ số của N

- Khởi tạo $t0 = 0$ để tích lũy kết quả số đảo và $t2 = 10$ làm cơ số thập phân.
- Lặp cho đến khi $N = 0$:
 - Lấy chữ số cuối: $t1 = N \% 10 = \text{digit}$.
 - Cập nhật số đảo: $reverse = reverse * 10 + \text{digit}$ (thực hiện qua mul và add trên t0).
 - Loại bỏ chữ số cuối: $N = N / 10$.
- Khi vòng lặp kết thúc, t0 chứa số đảo kết quả.

3. In kết quả

In chuỗi thông báo, chuyển giá trị từ t0 sang a0, rồi dùng syscall để in số đảo ra màn hình. Cuối cùng, kết thúc chương trình.

→ Chạy chương trình & quan sát kết quả:

Lưu ý: Với các test case có chữ số 0 ở cuối, khi đảo ngược các số 0 này sẽ không còn xuất hiện trong kết quả. Nguyên nhân là do chương trình xử lý dữ liệu dưới dạng **số nguyên**, nên các số 0 ở đầu của số đảo không có ý nghĩa và tự động bị loại bỏ.

Test case 1:



Messages Run I/O

Nhap N: 102
So dao nguoc la: 201

Test case 2:



Messages Run I/O

Nhap N: 123456789
So dao nguoc la: 987654321

Test case 3:



Messages Run I/O

Nhap N: 1200
So dao nguoc la: 21

Test case 4:



Messages Run I/O

Nhap N: 10
So dao nguoc la: 1

Test case 5:



Messages Run I/O

Nhap N: 12321
So dao nguoc la: 12321

→ Chương trình chạy đúng!

Assignment B:

13. Nhập mảng số nguyên từ bàn phím. In ra màn hình số các phần tử có giá trị khác nhau trong mảng.

→ Yêu cầu đề bài:

- Nhập mảng số nguyên từ bàn phím, ví dụ: {1, 3, 2, 4, 5, 3, 2, 1}
- In ra màn hình số các phần tử có giá trị khác nhau trong mảng: 5

→ Phân tích cách thực hiện:

- **Ý tưởng:** Để đếm số **giá trị khác nhau** trong mảng, ta xét **từng phần tử** và kiểm tra xem nó **đã xuất hiện trước đó chưa**.
- Với mỗi phần tử $a[i]$:
 - So sánh $a[i]$ với **tất cả** các phần tử đứng trước nó $a[0] \rightarrow a[i-1]$.
 - Nếu **tồn tại** một phần tử bằng $a[i] \rightarrow$ kết luận **đã xuất hiện, không tính**.
 - Nếu **không có** phần tử nào trùng $\rightarrow$ đây là **giá trị mới, tăng biến đếm**.
- Quá trình này cần **hai vòng lặp lồng nhau**:
 - Vòng ngoài duyệt từng phần tử theo chỉ số i .
 - Vòng trong dùng chỉ số j để so sánh với các phần tử trước đó.
- **Tóm lại:** So sánh $a[i]$ với $a[0..i-1]$: Có trùng $\rightarrow$ Bỏ qua || Không trùng $\rightarrow$ count++

→ Code + giải thích:

```
.data
arr: .space 400
# cấp phát bộ nhớ cho mảng (tối đa 100 phần tử, mỗi số nguyên chiếm 4 byte trong bộ nhớ)
msg1: .asciz "Nhập n: " # chuỗi thông báo nhập số phần tử n
msg2: .asciz "Nhập: "   # chuỗi thông báo nhập n phần tử
msg3: .asciz "Số giá trị khác nhau: " # chuỗi in kết quả

.text
main:
# nhập n
li a7, 4           # mã dịch vụ a7 = 4 → in chuỗi
la a0, msg1        # a0 = địa chỉ chuỗi msg1
ecall              # in "Nhập n: "
```

li a7, 5	# mã dịch vụ a7 = 5 → nhập số nguyên
ecall	# đọc n từ bàn phím → lưu n vào thanh ghi a0
mv s0, a0	# move giá trị n từ thanh ghi a0 → s0 hay s0 = n → lưu số phần tử

nhập mảng

la s1, arr	# s1 trở tới đầu mảng
li t0, 0	# t0 = i = 0

nhap:

beq t0, s0, dem # nếu i = n chuyển sang nhãn **dem** → thực hiện đếm số phần tử khác nhau
 # nếu không thì thực hiện câu lệnh bên dưới:

li a7, 4	# mã dịch vụ a7 = 4 → in chuỗi ra màn hình
la a0, msg2	# s0 = địa chỉ chuỗi msg2
ecall	# in "Nhap: "

li a7, 5	# mã dịch vụ a7 = 5 → nhập số nguyên
ecall	# đọc số nguyên từ bàn phím → lưu vào thanh ghi a0

sw a0, 0(s1)	# lưu vào arr[i]
addi s1, s1, 4	# con trỏ trở sang phần tử kế tiếp (mỗi int 4 byte)
addi t0, t0, 1	# i++
j nhap	# lặp lại bước nhập các phần tử trong mảng

Đếm số giá trị khác nhau

dem:

li s2, 0	# khởi tạo s2 = count = 0
li t0, 0	# i = 0
la s1, arr	# s1 trở tới đầu mảng

loop_ngoai:

beq t0, s0, in_kq	# nếu i == n → nhảy tới nhãn in_kq (in kết quả)
lw t1, 0(s1)	# t1 = arr[i]
la t3, arr	# t3 trở arr[0]
li t2, 0	# t2 = j = 0

loop_trong:

beq t2, t0, tang	# nếu j == i → chưa trùng → tăng count
# nếu không thì xét tiếp câu lệnh bên dưới:	
lw t4, 0(t3)	# t4 = arr[j]
beq t1, t4, boqua	# nếu trùng → nhảy tới nhãn bỏ qua (không tăng count)
# nếu chưa trùng thì tăng j xét tiếp	
addi t3, t3, 4	# trở sang arr[j+1]
addi t2, t2, 1	# j++
j loop_trong	# tiếp tục so sánh (thực hiện vòng for trong)

tang:

addi s2, s2, 1	# count++
----------------	-----------

```

boqua:
    addi s1, s1, 4      # trở sang arr[i+1]
    addi t0, t0, 1      # i++
    j loop_ngoai        # quay lại vòng for ngoài

# In kết quả
in_kq:
    li a7, 4            # mã dịch vụ a7 = 4 → in chuỗi ra màn hình
    la a0, msg3          # a0 = địa chỉ chuỗi msg3
    ecall               # in thông báo

    li a7, 1            # mã dịch vụ a7 = 1 → in số nguyên
    mv a0, s2            # s2 = count → truyền giá trị vào thanh ghi a0
    ecall               # in count ra màn hình

    li a7, 10           # mã dịch vụ a7 = 10 + ecall → kết thúc chương trình
    ecall

```

→ Tổng quan chương trình: Chương trình được chia thành **3 bước chính**:

1. Nhập dữ liệu

- In thông báo yêu cầu nhập số phần tử n.
- Dùng syscall để đọc n từ bàn phím và lưu vào thanh ghi.
- Dùng vòng lặp để nhập lần lượt n số nguyên và lưu vào mảng trong bộ nhớ.

2. Đếm số giá trị khác nhau trong mảng

- Khởi tạo biến đếm count = 0.
- Duyệt từng phần tử của mảng theo chỉ số i.
- Với mỗi a[i], dùng một vòng lặp khác với chỉ số j để so sánh với các phần tử đứng trước đó a[0..i-1].
 - Nếu phát hiện **trùng**, bỏ qua không tăng biến đếm.
 - Nếu **không trùng với bất kỳ phần tử nào trước đó**, tăng count.
- Kết thúc vòng lặp, biến count chứa số lượng giá trị khác nhau.

3. In kết quả

- In chuỗi thông báo ra màn hình.
- Đưa giá trị count vào thanh ghi in và dùng syscall để hiển thị kết quả.
- Kết thúc chương trình.

→ Chạy chương trình & quan sát kết quả:

Test case 1:

```
Messages Run I/O
Nhap n: 5
Nhap: 1
Nhap: 2
Nhap: 3
Nhap: 2
Nhap: 1
So gia tri khac nhau: 3
-- program is finished running (0) --
```

Test case 2:

```
Messages Run I/O
Nhap n: 5
Nhap: 1
Nhap: 1
Nhap: 1
Nhap: 1
Nhap: 1
So gia tri khac nhau: 1
-- program is finished running (0) --
```

Test case 3:

```
Messages Run I/O
Nhap n: 9
Nhap: 1
Nhap: 2
Nhap: 3
Nhap: 4
Nhap: 5
Nhap: 6
Nhap: 7
Nhap: 8
Nhap: 9
So gia tri khac nhau: 9
```

→ Chương trình chạy đúng!

Assignment C:

1. Nhập vào một chuỗi ký tự, kiểm tra đó có phải là một chuỗi đối xứng hay không? Ví dụ: "abc121cba" là một chuỗi đối xứng.

→ Yêu cầu đề bài:

- Nhập vào một chuỗi ký tự từ bàn phím.
- Kiểm tra xem chuỗi đó có phải là một chuỗi đối xứng hay không?
- Chuỗi đối xứng: đọc từ trái → phải và đọc từ phải → trái giống hệt nhau.
- Ví dụ: abcba, a, 121, abba...

→ Phân tích cách thực hiện:

- Chuỗi ký tự được lưu liên tiếp nhau trong bộ nhớ: mỗi ký tự chiếm 1 byte, kết thúc bằng ký tự '\0'.
- Ý tưởng:
 - Tìm độ dài chuỗi.
 - Đặt 2 con trỏ: **left** trỏ đầu chuỗi, **right** trỏ cuối chuỗi (trước '\0')
 - So sánh:
 - Nếu ***left != *right** (ký tự tại left != ký tự tại right) → không đối xứng.
 - Nếu bằng nhau → dịch **left++**, **right--** → thu hẹp phạm vi.
 - Lặp đến khi **left >= right** → là chuỗi đối xứng.

→ Code + giải thích:

```
.data
thongbao: .asciz "Nhap xau: " # Chuỗi thông báo nhập xâu
yes_msg: .asciz "La xau doi xung!\n" # Chuỗi thông báo khi xâu là đối xứng
no_msg: .asciz "Khong phai xau doi xung!\n" # Chuỗi thông báo khi xâu không đối xứng
buffer: .space 100
# cấp phát 100 byte để lưu chuỗi nhập (tối đa 99 ký tự thực + 1 byte cho '\0')

.text
main:
    # In thông báo nhập xâu
    li a7, 4 # mã dịch vụ a7 = 4 → in chuỗi ra màn hình
    la a0, thongbao # a0 chứa địa chỉ chuỗi thongbao
    ecall # thực hiện in thông báo "Nhap xau: "
```

Đọc chuỗi vào buffer

li a7, 8 # mã dịch vụ a7 = 8 → đọc chuỗi từ bàn phím
la a0, buffer # a0 chứa địa chỉ vùng nhớ lưu chuỗi
li a1, 100 # a1 = kích thước tối đa được nhập (tối đa 99 ký tự + 1 chỗ cho ký tự null)
ecall # thực hiện nhập chuỗi vào buffer

BƯỚC XÓA KÝ TỰ '\n' DO PHÍM ENTER

mv t5, a0 # t5 trở vào đầu chuỗi để xử lý

remove_newline:

lb t2, 0(t5) # đọc 1 byte ký tự tại vị trí t5
beq t2, zero, done_remove # nếu gặp '\0' thì dừng xóa → nhảy tới nhãn **done_remove**
li t3, 10 # t3 = 10 (mã ASCII của '\n')
beq t2, t3, set_null # nếu ký tự là '\n' thì nhảy tới nhãn **set_null** để thực hiện thay thế
nếu ký tự khác '\n' thì thực hiện tiếp câu lệnh bên dưới:
addi t5, t5, 1 # tăng con trỏ sang ký tự tiếp theo
j remove_newline # lặp lại quá trình

set_null:

sb zero, 0(t5) # ghi '\0' đè lên '\n'

done_remove:

BƯỚC TÌM ĐỘ DÀI CHUỖI (KHÔNG TÍNH '\0')

mv t5, a0 # t5 lại trở về đầu chuỗi để tính độ dài
li t4, 0 # t4 = length = 0

find_length:

lb t2, 0(t5) # đọc từng ký tự
beq t2, zero, done_length # nếu gặp '\0' thì dừng đếm → nhảy tới nhãn **done_length**
addi t4, t4, 1 # tăng độ dài lên 1 → length++
addi t5, t5, 1 # sang ký tự tiếp theo
j find_length # lặp lại quá trình

done_length:

BƯỚC KIỂM TRA ĐỐI XỨNG (2 CON TRỎ)

mv t0, a0 # t0 = left pointer (đầu chuỗi)
add t1, a0, t4 # t1 = a0 + length → trỏ tới ký tự null sau ký tự cuối cùng
addi t1, t1, -1 # lùi 1 byte → trỏ về ký tự cuối → right pointer (cuối chuỗi)

```

check_loop:
    bge t0, t1, is_palindrome
    # nếu left >= right → là xâu đối xứng → chuyển tới nhãn is_palindrome
    # nếu không thì thực hiện tiếp câu lệnh bên dưới!

    lb t2, 0(t0)      # lấy ký tự bên trái (*left)
    lb t3, 0(t1)      # lấy ký tự bên phải (*right)

    bne t2, t3, not_palindrome
    # nếu t2 != t3 → không đối xứng → chuyển tới nhãn not_palindrome
    # nếu t2 == t3 → thu hẹp phạm vi và xét tiếp!!!
    addi t0, t0, 1     # left++
    addi t1, t1, -1    # right--
    j check_loop      # tiếp tục vòng lặp check_loop

is_palindrome:
    li a7, 4           # mã dịch vụ a7 = 4 → in chuỗi ra màn hình
    la a0, yes_msg     # a0 chứa địa chỉ chuỗi yes_msg
    ecall              # in thông báo “La xau doi xung!\n” ra màn hình
    j end_program      # nhảy tới nhãn end_program

not_palindrome:
    li a7, 4           # mã dịch vụ a7 = 4 → in chuỗi ra màn hình
    la a0, no_msg      # a0 chứa địa chỉ chuỗi no_msg
    ecall              # in thông báo “Khong phai xau doi xung!\n” ra màn hình

end_program:
    li a7, 10          # mã dịch vụ a7 = 10 + ecall → kết thúc chương trình!
    ecall

```

Suport: Giải thích tại sao cần phải loại bỏ dấu ‘\n’:

- Đoạn **code nhập xâu** trong chương trình:

```

.data
buffer: .space 100
# cấp phát 100 byte để lưu chuỗi nhập (tối đa 99 ký tự thực + 1 byte chỉ ký tự '\0')

.text
main:
    # Đọc chuỗi vào buffer

    li a7, 8          # mã dịch vụ a7 = 8 → đọc chuỗi từ bàn phím

```

```

la a0, buffer    # a0 chứa địa chỉ vùng nhớ lưu chuỗi
li a1, 100
# a1 = kích thước tối đa được nhập (tối đa 99 ký tự + 1 chỗ cho ký tự '\0')
ecall # thực hiện nhập chuỗi vào buffer

```

- Giả sử ta nhập chuỗi “abcd” và quan sát trên Data Segment:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)
0x10010000	d c b a	\0 \0 \0 \n	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0x10010020	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0x10010040	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0x10010060	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0x10010080	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0x100100a0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0x100100c0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0

- Nhìn vào Data Segment ta thấy xâu vừa nhập được lưu trong bộ nhớ theo dạng:

```
'a' 'b' 'c' 'd' '\n' '\0'
```

- Khi nhập xâu từ bàn phím, sau khi gõ các ký tự và nhấn **Enter**, ký tự xuống dòng '\n' cũng được lưu vào bộ nhớ trước ký tự kết thúc chuỗi '\0'.
- Vì vậy chuỗi thực tế lưu trong RAM có dạng: '... \n \0'.
- Nếu không loại bỏ ký tự '\n' này trước khi xử lý, chương trình sẽ tính cả '\n' vào độ dài chuỗi và đem so sánh khi kiểm tra đối xứng, dẫn đến kết quả sai.

→ Tổng quan chương trình: Chương trình được chia thành **4 bước chính**:

1. Nhập chuỗi từ bàn phím

- In thông báo: “Nhập xâu: ”
- Dùng syscall đọc chuỗi vào vùng nhớ buffer (tối đa 99 ký tự + ký tự kết thúc '\0').
- Sau khi nhập, chuỗi trong buffer có dạng: "kytu...kytu\n\0"

Do khi nhấn **Enter**, hệ thống tự thêm ký tự **newline '\n'** trước '\0'.

2. Tiền xử lý chuỗi

Gồm 2 công việc quan trọng để chuỗi có thể kiểm tra chính xác:

a) Xóa ký tự '\n'

- Duyệt từng ký tự trong chuỗi.
- Nếu gặp mã ASCII 10 (newline), thay bằng '\0'.
- Mục đích: chuỗi trở thành đúng dạng: "kytu...kytu\0"

→ tránh việc ký tự '\n' làm sai kết quả so sánh.

b) Tính độ dài chuỗi

- Duyệt lại từ đầu chuỗi.
- Đếm số ký tự cho đến khi gặp '\0'.
- Lưu độ dài vào thanh ghi t4.

3. Kiểm tra đối xứng bằng kỹ thuật 2 con trỏ

Sử dụng hai con trỏ:

- t0 trỏ vào **đầu chuỗi** (left pointer).
- t1 trỏ vào **cuối chuỗi** (right pointer).

Quy trình lặp:

- So sánh ký tự tại left và right.
- Nếu **khác nhau** → kết luận **không đối xứng**.
- Nếu **giống nhau** → thu hẹp phạm vi:
 - left++
 - right--
- Khi left >= right → kết luận **xâu đối xứng**.

4. In kết quả và kết thúc

- Nếu đối xứng → in: La xau doi xung!
- Nếu không → in: Khong phai xau doi xung!
- Kết thúc chương trình bằng syscall.

➔ Chạy chương trình & quan sát kết quả:

Test case 1: nhập xâu không đối xứng bất kỳ



The screenshot shows a window with two tabs: "Messages" and "Run I/O". The "Messages" tab is selected. The output text is as follows:

```
Nhap xau: abcd
Khong phai xau doi xung!

-- program is finished running (0) --
```

Test case 2: nhập 1 ký tự

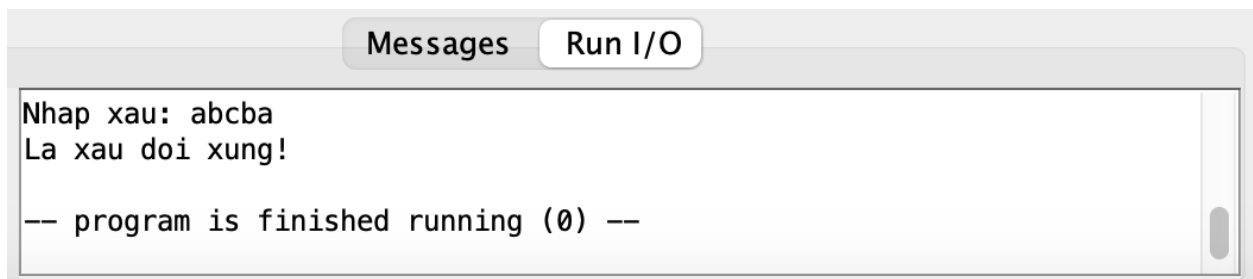


The screenshot shows a window with two tabs: "Messages" and "Run I/O". The "Messages" tab is selected. The output text is as follows:

```
Nhap xau: a
La xau doi xung!

-- program is finished running (0) --
```

Test case 3: nhập xâu đối xứng có độ dài lẻ



The screenshot shows a window with two tabs: "Messages" and "Run I/O". The "Messages" tab is selected. The output text is as follows:

```
Nhap xau: abcba
La xau doi xung!

-- program is finished running (0) --
```

Test case 4: nhập xâu đối xứng có độ dài chẵn



The screenshot shows a window with two tabs: "Messages" and "Run I/O". The "Messages" tab is selected. The output text is as follows:

```
Nhap xau: 0000
La xau doi xung!

-- program is finished running (0) --
```

Test case 5: nhập xâu rỗng



→ Chương trình chạy đúng!